

OpenZeppelin 첫 사용

Ownable · Pausable · AccessControl

감사된 코드로 접근 제어를 구현한다

Remix IDE

OZ v5

Phase 1 기반

OpenZeppelin이란

S34에서 직접 만든 것 vs OpenZeppelin이 제공하는 것 · 1/3

스마트컨트랙트 표준 구현체 라이브러리 — ERC-20·ERC-721·ERC-1155, 접근 제어, 업그레이드 패턴을 감사(audit)된 코드로 제공한다

S34에서 직접 만든 것

- `modifier onlyOwner()`
- 단순 owner 주소 비교
- 이벤트 없음
- 소유권 이전 없음
- 엡지케이스 미처리

OpenZeppelin이 제공하는 것

- **Ownable** — onlyOwner + transferOwnership + 엡지케이스
- **Pausable** — whenNotPaused + _pause/_unpause
- **AccessControl** — 역할 기반 + 역할별 admin + 이벤트
- 수십 개 외부 감사 + 버그바운티 검증

원칙: 상속받아 그대로 쓰는 것이 직접 구현보다 안전하다. OZ 코드를 수정해서 쓰는 것 자체가 위험하다.

버전: 이 강의는 **OZ v5 기준** — v4와 생성자 시그니처·에러 형태가 다름

왜 직접 짜지 않는가?

역사적 사고 — 직접 구현한 권한 로직의 실수 · 2/3

사건	연도	손실	원인
The DAO 해킹	2016	약 \$60M	재진입(reentrancy) — 직접 구현한 자금 인출 로직
Parity Wallet 1차	2017.7	약 \$32M 도난	delegatecall + 초기화 로직 미구현 (직접 만든 멀티시그)
Parity Wallet 2차	2017.11	~513,743 ETH 동결	selfdestruct 접근 제어 미적용, 라이브러리 컨트랙트 소각
Bancor 해킹	2018	약 \$23.5M	직접 구현한 전송 함수의 접근 제어 버그

세 사고의 공통점: **직접 작성한 접근 제어 권한 코드의 실수** — OpenZeppelin이 이미 해결한 문제들이다

OZ 코드의 신뢰 근거: Trail of Bits · Certora · Sigma Prime 등 수십 개 독립 감사 · 수백만 달러 버그바운티 · 수천 개 프로덕션 컨트랙트 의존

수치 주의: Parity 2차 동결액은 시점에 따라 \$150M~\$280M으로 다르게 인용됨. 동결 시점 기준 약 \$150M이 정확.

OpenZeppelin import 방법

Remix / Hardhat 프로젝트 · 3/3

Remix에서 import

```
// @openzeppelin 경로 →  
// Remix가 자동으로 GitHub fetch  
import "@openzeppelin/contracts/access/Ownable.sol";  
import "@openzeppelin/contracts/utils/Pausable.sol";  
import "@openzeppelin/contracts/access/AccessControl.sol";
```

별도 설치 없이 바로 사용 — Remix가 자동 다운로드

Hardhat 프로젝트에서 import

```
# 1. 패키지 설치  
npm install @openzeppelin/contracts  
  
// 2. Solidity에서 동일하게 import  
import "@openzeppelin/contracts/access/Ownable.sol";
```

node_modules에 설치 후 import — 경로는 동일

contracts vs contracts-upgradeable: 프록시(업그레이드) 패턴에는 `@openzeppelin/contracts-upgradeable` 을 사용한다. Phase 1은 일반 `@openzeppelin/contracts` . M9 업그레이드 패턴에서 전환 예정.

OZ v4 vs v5: v5는 `Ownable(address initialOwner)` 생성자 인자 필수. v4 튜토리얼 참고 시 주의.

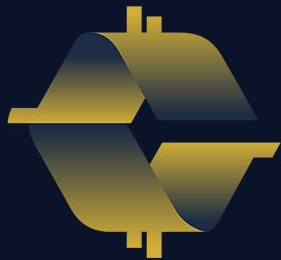
01 Ownable

단일 소유자 패턴 · Ownable2Step · 한계

onlyOwner

transferOwnership

Ownable2Step



COINCRAFT
ACADEMY

Ownable — 단일 소유자 패턴

contract MyContract is Ownable · 1/4

가장 단순한 접근 제어. owner 1명이 관리하는 컨트랙트에 사용한다.

```
import "@openzeppelin/contracts/access/Ownable.sol";

contract MyContract is Ownable {
    // OZ v5: Ownable 생성자에 initialOwner 필수
    constructor(address initialOwner) Ownable(initialOwner) {}

    function sensitiveAction() public onlyOwner {
        // owner만 호출 가능
    }
}
```

Ownable(initialOwner) — 외부 주소 받기

- 배포자가 아닌 다른 주소를 초기 owner로 지정 가능
- 예: 멀티시그(Gnosis Safe)를 owner로 배포
- 유연성 필요 시 권장

Ownable(msg.sender) — 배포자 고정

- 배포자가 자동으로 owner
- 간단한 프로젝트에 적합
- 나중에 transferOwnership으로 이전 가능

두 형태 모두 유효 — 핵심은 **address(0)**은 안 된다 (v5: OwnableInvalidOwner custom error)

Ownable이 제공하는 것

함수 · modifier · 이벤트 · 2/4

함수 / modifier	설명
<code>owner()</code>	현재 소유자 주소 반환
<code>onlyOwner</code>	<code>owner</code> 가 아니면 revert → v5: <code>OwnableUnauthorizedAccount</code> custom error
<code>transferOwnership(newOwner)</code>	소유권 즉시 이전 — 오타 시 복구 불가 ⚠
<code>renounceOwnership()</code>	소유권 영구 포기 (<code>address(0)</code> 설정) — 되돌릴 수 없음
<code>OwnershipTransferred</code> 이벤트	소유권 변경 시 온체인 기록

transferOwnership의 치명적 문제: 호출 즉시 소유권 이전. 잘못된 주소 입력 시 컨트랙트 영구 lock

사례: `transferOwnership("0xDEAD...beef")` → 오타 → 아무도 컨트롤 불가 → 수억 원 자산 동결

v4 vs v5 차이: v4 `onlyOwner` 실패 시 문자열 에러 "Ownable: caller is not the owner"

v5: custom error `OwnableUnauthorizedAccount(account)` — Remix Logs에서 형태가 다름

Ownable2Step — 소유권 이전 사고 방지

2단계 이전으로 오타·실수 방지 · 3/4

문제: `transferOwnership` 은 즉시 이전 → 잘못된 주소 입력 시 컨트랙트 영구 lock

```
import "@openzeppelin/contracts/access/Ownable2Step.sol";

contract SafeContract is Ownable2Step {
    constructor() Ownable(msg.sender) {} // Ownable2Step도 Ownable 생성자 호출
}

// 1단계: transferOwnership(newOwner) → pendingOwner 설정 (즉시 이전 안 됨)
// 2단계: newOwner가 acceptOwnership() 직접 호출해야 완료
// 취소: 현재 owner가 transferOwnership(currentOwner) 다시 호출
```

Ownable (1단계)

- `transferOwnership` → 즉시 완료
- 수락 과정 없음
- 오타 = 영구 lock

Ownable2Step (2단계)

- 1단계: `pendingOwner` 설정만
- 2단계: 새 주소가 직접 `acceptOwnership()`
- 새 주소 접근 불가 → 자동 실패 → 안전

결론: 중요한 자산을 관리하는 컨트랙트는 항상 `Ownable2Step`을 사용하라

Ownable의 한계

단일 소유자 구조의 한계 · 4/4

핵심 문제: 소유자가 1명뿐이다 — 역할을 분리할 수 없다

교보생명 시스템의 요구사항

- NFT 발행은 VASP 서버만 가능해야 한다
- 긴급 정지는 운영팀 EOA가 독립적으로 실행할 수 있어야 한다
- 감사 조회는 감사팀만 가능해야 한다
- 한 명이 키를 잃거나 탈취당해도 전체 시스템이 마비되면 안 된다

	Ownable	AccessControl
역할 분리	불가	가능
복수 권한자	불가	역할당 N명
교보 시스템 적합	X	✓

결론: 교보 시스템처럼 발행·정지·감사가 분리되어야 하면 → AccessControl이 필요하다

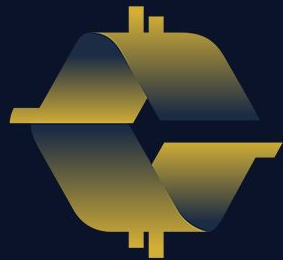
02

Pausable

긴급 정지 (Circuit Breaker) · _update 후

whenNotPaused

Phase 1 적용



COINCRAFT
ACADEMY

Pausable — 긴급 정지 (Circuit Breaker)

contract KyoboBase is Ownable, Pausable · 1/3

운영 중 사고(해킹 탐지·규제 요청·버그 발견)에 즉시 대응. 전기 회로의 차단기처럼 이상 신호 감지 시 전체 흐름을 차단한다.

```
import "@openzeppelin/contracts/utils/Pausable.sol";
import "@openzeppelin/contracts/access/Ownable.sol";

contract KyoboBase is Ownable, Pausable {
    constructor() Ownable(msg.sender) {}

    function pause() public onlyOwner { _pause(); }
    function unpause() public onlyOwner { _unpause(); }

    function mint(address to, uint256 amt) public onlyOwner whenNotPaused {
        // 정지 중 발행 불가
    }

    function balanceOf(address who) public view returns (uint256) {
        // 정지 중에도 조회는 가능 (whenNotPaused 없음)
    }
}
```

주의: `_pause()` 는 internal — 외부 노출 시 반드시 권한 modifier(`onlyOwner` / `onlyRole`) 필수. 안 붙이면 누구나 컨트랙트를 멈출 수 있다.

Pausable이 제공하는 것

modifier · 함수 · 이벤트 · 2/3

modifier / 함수	설명
<code>whenNotPaused</code>	<code>paused</code> 이면 revert — v5: <code>EnforcedPause()</code> / v4: <code>"Pausable: paused"</code>
<code>whenPaused</code>	<code>paused</code> 일 때만 통과 — v5: <code>ExpectedPause()</code>
<code>_pause()</code>	내부 정지 상태 전환 (internal) — 외부 노출 시 권한 modifier 필수
<code>_unpause()</code>	내부 재개 상태 전환 (internal)
<code>paused()</code>	현재 정지 상태 조회 (true/false)
<code>Paused</code> / <code>Unpaused</code> 이벤트	상태 변경 시 온체인 기록

pause가 필요한 실제 상황

- 해킹 탐지 — 추가 피해 즉시 차단
- 금감원·금융위 규제 요청
- 오라클 장애 — 잘못된 가격 반영 방지
- 컨트랙트 업그레이드 준비

내부 구조 — 단순한 bool 하나

```
bool private _paused; // 단순 bool

// v5 custom error
error EnforcedPause();
error ExpectedPause();
```

단순한 bool 하나로 전체 시스템 특정 기능을 정지 — ISMS·금융감독원 보안 요건에서도 요구하는 기능

KyoboNFT에서 _update 혹은 선택한 이유

ERC-1155 단일 진입점 제어 + Phase 1 적용 · 3/3

_update란? ERC-1155의 mint · transfer · burn 모두의 공통 진입점 — v5에서 _beforeTokenTransfer가 _update로 통합됨

❌ 개별 함수에 modifier 달기

```
function mint(...) whenNotPaused { ... }  
// 문제: safeTransferFrom은 paused  
// 상태에서도 동작함 → 우회 가능
```

✅ _update 혹은 한 곳에 달기

```
function _update(  
    address from, address to,  
    uint256[] memory ids,  
    uint256[] memory values  
) internal virtual override whenNotPaused {  
    super._update(from, to, ids, values);  
}
```

결론: _update 한 곳에만 whenNotPaused → mint·transfer·burn 전체 통제. 우회 불가.

Phase 1 적용: 규제 기관 요청·해킹 탐지 시 pause() 호출 → 모든 토큰 이동 즉시 차단. 정지 중에는 mint() 포함 모든 이동 revert.

03

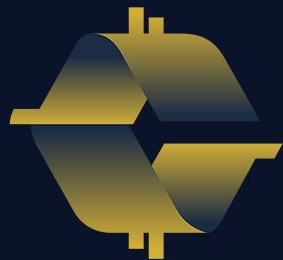
AccessControl

역할 기반 접근 제어 · 역할 체계 설계 · AccessControlEnumerable

역할(Role)

bytes32

Phase 1 설계



COINCRAFT
ACADEMY

AccessControl — 역할 기반 접근 제어

Ownable 한계 극복 · bytes32 role · 충돌 · 1/3

Ownable의 "1인 소유자" 한계를 극복한다. **역할(Role)**을 정의하고 여러 주소에 부여한다.

내부 핵심 구조

```
struct RoleData {  
    mapping(address => bool) hasRole;  
    bytes32 adminRole;  
}  
mapping(bytes32 role => RoleData) private _roles;
```

- **hasRole**: 주소 → 역할 보유 여부
- **adminRole**: 이 역할을 관리하는 상위 역할

role ID가 bytes32인 이유

- `keccak256("MINTER_ROLE")` → 32바이트 고정 해시
- 문자열 비교는 gas 多, bytes32 비교는 단일 EVM 연산
- 충돌 확률: 2^{256} 분의 1 → 사실상 0
- 함수 선택자(4바이트 자름)와 달리 전체 **256비트** 사용

충돌에 대하여: role ID는 keccak256 출력 전체(256비트)를 쓴다. 서로 다른 역할 문자열이 같은 ID를 가질 확률은 사실상 0 — 함수 선택자(앞 4바이트만)와 완전히 다름.

DEFAULT_ADMIN_ROLE = `0x00` (32바이트 전부 0) — 모든 역할의 기본 admin

AccessControl — contract KyoboNFTController

역할 정의 · 부여 · 사용 패턴 · 2/3

```
import "@openzeppelin/contracts/access/AccessControl.sol";

contract KyoboNFTController is AccessControl {
    bytes32 public constant MINTER_ROLE = keccak256("MINTER_ROLE");
    bytes32 public constant PAUSER_ROLE = keccak256("PAUSER_ROLE");
    bytes32 public constant AUDITOR_ROLE = keccak256("AUDITOR_ROLE");

    constructor() {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender); // 체크 없이 강제 부여
        _grantRole(MINTER_ROLE, msg.sender);
    }

    function mint(address to, uint256 tokenId, uint256 amount)
        public onlyRole(MINTER_ROLE) { ... }

    function pause() public onlyRole(PAUSER_ROLE) { ... }
    function getAuditLog() public view onlyRole(AUDITOR_ROLE) returns (bytes memory) { ... }
}
```

_grantRole vs grantRole — 핵심 구분:

- `_grantRole` (internal): 권한 검사 없이 부여 → constructor 초기화 전용
- `grantRole` (external): 해당 역할의 adminRole 보유자만 호출 가능 → 운영 중 사용
- ⚠ 운영 코드에서 `_grantRole` 을 외부 함수로 호출하면 권한 검사를 건너뛰는 보안 구멍

AccessControl이 제공하는 것

함수 · modifier · 이벤트 · 3/3

함수 / modifier	설명
<code>onlyRole(role)</code>	역할 없으면 revert — v5: <code>AccessControlUnauthorizedAccount(account, neededRole)</code>
<code>hasRole(role, account)</code>	역할 보유 여부 조회 (bool)
<code>grantRole(role, account)</code>	역할 부여 — 해당 역할의 adminRole 보유자만 가능
<code>revokeRole(role, account)</code>	역할 회수 — adminRole 보유자만
<code>renounceRole(role, callerConfirmation)</code>	자기 역할 스스로 반납 — v5: 본인 주소 확인 인자 필요
<code>getRoleAdmin(role)</code>	그 역할을 부여·회수할 수 있는 관리 역할의 bytes32 반환
<code>RoleGranted</code> / <code>RoleRevoked</code> 이벤트	역할 변경 시 온체인 기록 → 감사 추적

v5 에러 형태: v4의 긴 문자열 `"AccessControl: account 0x... is missing role 0x..."` 대신

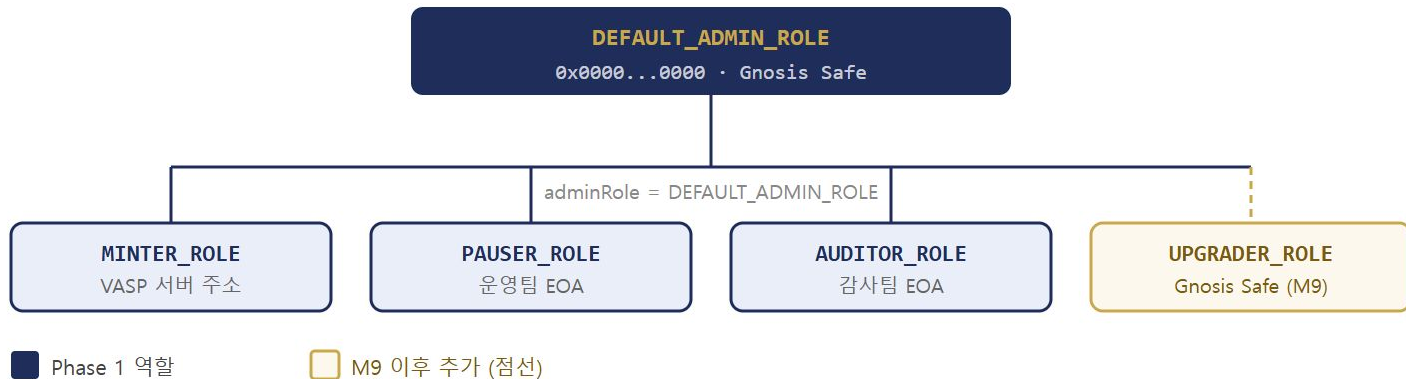
`custom error` `AccessControlUnauthorizedAccount(account, neededRole)` — Remix Logs에서 이 형태로 보임

이벤트 = 감사 기록: `RoleGranted`·`RoleRevoked` 이벤트가 모든 역할 변경을 온체인에 기록 → 규제 감사 대응

AccessControl 역할 체계 설계

기본 역할 구조 · 1/4

역할을 부여·회수할 수 있는 **adminRole**을 명시적으로 설정하지 않으면 모든 역할의 admin은 DEFAULT_ADMIN_ROLE(0x00)이 된다



역할별 admin 커스터마이징

올바른 패턴 — `_setRoleAdmin()` 명시 설정 · 2/4

함정: `_setRoleAdmin()` 을 빠뜨리면 선언한 역할 계층이 실제로 동작하지 않는다 — 여전히 DEFAULT_ADMIN만 관리 가능

```
bytes32 public constant ADMIN_ROLE    = keccak256("ADMIN_ROLE");
bytes32 public constant MINTER_ROLE   = keccak256("MINTER_ROLE");
bytes32 public constant PAUSER_ROLE   = keccak256("PAUSER_ROLE");

constructor(address admin_address) {
    _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
    _grantRole(ADMIN_ROLE, admin_address);

    // 핵심: adminRole 명시적 설정 - 이 줄을 빠뜨리면 ADMIN_ROLE이 MINTER 관리 못 함
    _setRoleAdmin(MINTER_ROLE, ADMIN_ROLE); // ADMIN_ROLE이 MINTER_ROLE 부여·회수
    _setRoleAdmin(PAUSER_ROLE, ADMIN_ROLE);  // ADMIN_ROLE이 PAUSER_ROLE 부여·회수
}
```

설정 후 역할 계층

DEFAULT_ADMIN_ROLE

- └ ADMIN_ROLE 부여·회수 가능
 - └ MINTER_ROLE 부여·회수 가능
 - └ PAUSER_ROLE 부여·회수 가능

설정 없을 때 (함정)

DEFAULT_ADMIN_ROLE

- └ MINTER_ROLE 부여·회수 (여전히)
 - └ PAUSER_ROLE 부여·회수 (여전히)
- ADMIN_ROLE → MINTER 관리 불가

역할 설계 3대 실수

안티패턴 — 반드시 피해야 할 것들 · 3/4

실수	증상	올바른 해결
실수 1 DEFAULT_ADMIN_ROLE → address(0) 부여	역할 관리 영구 불가 컨트랙트 접근 제어 locked	Gnosis Safe 등 실제 주소에 부여. address(0) 절대 금지
실수 2 deployer EOA가 모든 역할 보유한 채 운영 시작	EOA 키 탈취 시 전체 권한 장악 grantRole → revokeRole → 시스템 파괴	배포 직후 Gnosis Safe로 이전 후 운영 시작. deployer revoke 필수
실수 3 _setRoleAdmin() 빠뜨림	ADMIN_ROLE이 MINTER 관리 못 함 역할 계층이 실제로 동작 안 함	constructor에서 반드시 명시적 설정

실수 2 공격 시나리오:

공격자 키 탈취 → grantRole(MINTER_ROLE, attacker) → 대량 mint → revokeRole(DEFAULT_ADMIN_ROLE, deployer) → 복구 차단

Phase 1 역할 설계

교보생명 KyoboNFT 역할 매트릭스 · 4/4

역할	보유자	가능한 작업	비고
DEFAULT_ADMIN_ROLE	Gnosis Safe (멀티시그)	전체 역할 부여·회수	3-of-5 서명
MINTER_ROLE	VASP 서버 주소	NFT 발행	서버 교체 시 교체
PAUSER_ROLE	운영팀 EOA	긴급 정지·재개	단일 서명, 즉시
UPGRADER_ROLE	Gnosis Safe	컨트랙트 업그레이드	M9 이후 활성화

배포 시 올바른 순서:

- ① `_grantRole(DEFAULT_ADMIN_ROLE, deployer)` ← constructor
- ② `_grantRole(MINTER_ROLE, VASP_ADDRESS)`
- ③ `grantRole(DEFAULT_ADMIN_ROLE, GNOSIS_SAFE)` ← Gnosis Safe 먼저
- ④ `revokeRole(DEFAULT_ADMIN_ROLE, deployer)` ← deployer 마지막에 회수 ⚠

순서 절대 준수: ③ Gnosis Safe에 DEFAULT_ADMIN 부여 전에 ④ deployer를 revoke하면 → 아무도 admin 없는 상태 → 컨트랙트 영구 잠김

Ownable vs AccessControl — 선택 기준

언제 무엇을 쓰는가

	Ownable	Ownable2Step	AccessControl
소유자 수	1명	1명	역할당 N명
역할 분리	불가	불가	가능
이전 안전성	즉시 이전 ⚠	수락 필요 ✅	N/A
복잡도	낮음	낮음	높음
적합한 상황	개인 프로젝트	중요 자산 단일 관리자	기업 시스템, 역할 분리
KyoboNFT	X	X	✓

Ownable 선택 기준

- 개인 지갑이나 단순 컨트랙트
- owner 1명으로 충분한 경우
- 빠른 프로토타이핑

AccessControl 선택 기준

- 기업 시스템 — 역할 분리 필요
- 발행·정지·감사 역할이 다를 때
- 여러 주소에 동일 역할 부여 필요

AccessControlEnumerable

역할 멤버 조회 — contract KyoboNFT · 1/2

표준 AccessControl의 한계: `_roles[role].hasRole[account]` 매핑은 단방향 (주소→bool). "MINTER_ROLE을 가진 주소가 몇 명인지" 온체인 조회 불가

```
import "@openzeppelin/contracts/access/extensions/AccessControlEnumerable.sol";

contract KyoboNFT is AccessControlEnumerable {
    bytes32 public constant MINTER_ROLE = keccak256("MINTER_ROLE");

    constructor() { _grantRole(DEFAULT_ADMIN_ROLE, msg.sender); }

    // 추가 함수: MINTER_ROLE 보유 주소 전체 조회
    function getMinters() public view returns (address[] memory) {
        uint256 count = getRoleMemberCount(MINTER_ROLE);
        address[] memory minters = new address[](count);
        for (uint256 i = 0; i < count; i++) {
            minters[i] = getRoleMember(MINTER_ROLE, i);
        }
        return minters;
    }
}
```

추가된 함수: `getRoleMemberCount(role)` — 역할 보유자 수 | `getRoleMember(role, index)` — index번째 보유자 주소

AccessControlEnumerable — 언제 쓰는가

트레이드오프 · 2/2

Enumerable를 써야 하는 경우

- 관리자 UI에서 "현재 MINTER_ROLE 보유자 목록" 표시 필요
- 감사 보고서용 역할 보유자 온체인 스냅샷 조회
- 이벤트 로그 대신 온체인 상태 직접 조회 필요
- 규제 감사 — "현재 발행 권한자 몇 명?"

기본 AccessControl로 충분한 경우

- 역할 보유자 목록 조회가 불필요
- gas 최적화가 중요할 때
- 이벤트 로그(RoleGranted/Revoked)로 목록 추적 가능

비교 항목	AccessControl	AccessControlEnumerable
grantRole gas	낮음	약간 더 높음 (목록 유지)
역할 멤버 열거	불가	가능
KyoboNFT Phase 1	—	선택

내부 구조: EnumerableSet은 `_values` 배열 + `_positions` 매핑 조합 → $O(1)$ 추가·삭제·존재 확인.
배열만 있으면 삭제 $O(n)$, 매핑만 있으면 열거 불가 — 두 자료구조를 동시에 유지.

Phase 1 전체 접근 제어 흐름

배포 시 · 1/2

배포 시 역할 설정 순서가 틀리면 컨트랙트가 영구 잠긴다. 순서 엄수.

배포 시:

① KyoboNFT 배포

deployer → DEFAULT_ADMIN_ROLE 자동 부여 (`_grantRole` in constructor)

② 운영 역할 부여

deployer → `grantRole(MINTER_ROLE, VASP_SERVER_ADDRESS)`

deployer → `grantRole(PAUSER_ROLE, OPS_TEAM_ADDRESS)`

③ Gnosis Safe에 최상위 권한 이전 (반드시 ② 전에)

deployer → `grantRole(DEFAULT_ADMIN_ROLE, GNOSIS_SAFE_ADDRESS)`

④ deployer 권한 회수 (반드시 마지막)

deployer → `revokeRole(DEFAULT_ADMIN_ROLE, deployer)`

⚠ ③ 완료 전에 ④ 실행 → 아무도 admin 없는 상태 → 영구 lock

결과 상태 (배포 완료)

- Gnosis Safe = DEFAULT_ADMIN_ROLE
- VASP 서버 = MINTER_ROLE
- 운영팀 EOA = PAUSER_ROLE
- deployer = 역할 없음

Gnosis Safe 특성

- 멀티시그 — 예: 3-of-5 서명
- 단일 키 탈취로 전체 장악 불가
- 역할 변경마다 다수결 필요
- M9에서 실제 연동 예정

Phase 1 전체 접근 제어 흐름

운영 중 · 2/2

운영 중:

[정상 발행]

VASP 서버 → `mint(to, tokenId, amount)`

KyoboNFT → `onlyRole(MINTER_ROLE)` 검증 → 통과 → 발행 + `NFTMinted` 이벤트

[이상 탐지 → 긴급 정지]

OPS_TEAM → `pause()`

KyoboNFT → `onlyRole(PAUSER_ROLE)` 검증 → `_paused = true`

이후 모든 `mint/transfer` → `_update()` → `whenNotPaused` → `EnforcedPause()` revert

[원인 해소 → 재개]

OPS_TEAM → `unpause()` → `_paused = false` → 정상 운영 재개

[VASP 서버 교체]

Gnosis Safe 3-of-5 서명 수집

→ `revokeRole(MINTER_ROLE, OLD_VASP_ADDRESS)`

→ `grantRole(MINTER_ROLE, NEW_VASP_ADDRESS)`

→ 이벤트: `RoleRevoked` + `RoleGranted` → 온체인 감사 기록 자동 생성

온체인 감사 추적: `RoleGranted`·`RoleRevoked`·`Paused`·`Unpaused`·`NFTMinted` 이벤트가 모든 권한 변경과 발행을 자동 기록 → 규제 대응 근거

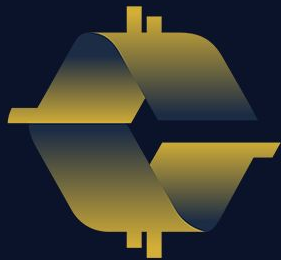
04 실습

KyoboMintController · 역할 계층 실험 · Ownable2Step 이전

실습 A

실습 B

실습 C



COINCRAFT
ACADEMY

실습 A — KyoboMintController 핵심 시나리오

AccessControl + Pausable 통합 · Remix · 5분

```
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/access/AccessControl.sol";
import "@openzeppelin/contracts/utils/Pausable.sol";

contract KyoboMintController is AccessControl, Pausable {
    bytes32 public constant MINTER_ROLE = keccak256("MINTER_ROLE");
    bytes32 public constant PAUSER_ROLE = keccak256("PAUSER_ROLE");
    uint256 public mintCount;
    event NFTMinted(address indexed to, uint256 indexed tokenId, uint256 amount);

    constructor() {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
        _grantRole(MINTER_ROLE, msg.sender);
        _grantRole(PAUSER_ROLE, msg.sender);
    }

    function mint(address to, uint256 tokenId, uint256 amount)
        public onlyRole(MINTER_ROLE) whenNotPaused {
        mintCount++; emit NFTMinted(to, tokenId, amount);
    }

    function pause() public onlyRole(PAUSER_ROLE) { _pause(); }
    function unpause() public onlyRole(PAUSER_ROLE) { _unpause(); }
}
```

확인 시나리오

- ① Account1 → `grantRole(MINTER_ROLE, Account2)`
- ② Account2 → `mint(Account3, 1, 100)` → NFTMinted 이벤트
- ③ Account1 → `pause()` → `paused() = true`
- ④ Account2 → `mint()` → `EnforcedPause revert` ✓
- ⑤ Account1 → `unpause()` → 다시 mint 성공
- ⑥ Account3 (역할 없음) → `mint()` → `AccessControlUnauthorizedAccount` ✓

실습 B — 역할 계층 실험

AccessControlEnumerable + _setRoleAdmin · Remix · 3분

```
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/access/extensions/AccessControlEnumerable.sol";

contract RoleHierarchyDemo is AccessControlEnumerable {
    bytes32 public constant ADMIN_ROLE = keccak256("ADMIN_ROLE");
    bytes32 public constant MINTER_ROLE = keccak256("MINTER_ROLE");

    constructor() {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
        _grantRole(ADMIN_ROLE, msg.sender);
        // MINTER_ROLE의 admin = ADMIN_ROLE (DEFAULT_ADMIN 아님)
        _setRoleAdmin(MINTER_ROLE, ADMIN_ROLE);
    }

    function getMinterCount() public view returns (uint256) {
        return getRoleMemberCount(MINTER_ROLE);
    }

    function getMinter(uint256 index) public view returns (address) {
        return getRoleMember(MINTER_ROLE, index);
    }
}
```

확인 시나리오

- ① Account1 → `grantRole(ADMIN_ROLE, Account2)`
- ② Account2(ADMIN) → `grantRole(MINTER_ROLE, Account3)` → 성공
- ③ `getMinterCount()` → 1
- ④ `getMinter(0)` → Account3 주소
- ⑤ Account3 → `grantRole(MINTER_ROLE, Account4)` → 실패

MINTER의 admin은 ADMIN_ROLE — Account3는 ADMIN 없음

실습 C — Ownable2Step 2단계 이전 시연

acceptOwnership 흐름 · Remix · 2분

```
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/access/Ownable2Step.sol";

contract TwoStepDemo is Ownable2Step {
    constructor() Ownable(msg.sender) {}

    function sensitiveAction() public onlyOwner {}
}
```

확인 시나리오

- ① Account1 → transferOwnership(Account2)
pendingOwner() = Account2, owner()는 아직 Account1
- ② owner() 확인 → Account1 (이전 미완료)
- ③ Account1 → sensitiveAction() → 성공
- ④ Account3 → acceptOwnership() → 실패
pendingOwner 아님 → OwnableUnauthorizedAccount
- ⑤ Account2 → acceptOwnership() → 성공
owner() = Account2
- ⑥ Account1 → sensitiveAction() → 실패

핵심: 새 주소(Account2)가 직접 acceptOwnership() 호출해야 완료 → 잘못된 주소 입력해도 안전

다음 세션 S37: ERC-20 직접 구현 — 오늘 배운 AccessControl + Pausable을 ERC-20 위에 얹는다